12 · Settings

Account, preferences, privacy and consent, support, health centre, and working hours.

New to the syntax? Read **00 · How to read this code** first; this guide explains what is specific to this feature.

What this feature does

Settings is one page with tabs. Citizens see **Account** (password, devices, deactivate), **Preferences** (theme, notifications), **Privacy** (consents, consent history, download your data), **Support** (raise a concern) and **Health centre** (switch centre). Workers also see **Work** (working hours, used to suggest visit times).

Consent history is append-only: a change adds a row, and the newest row is the current answer.

Files involved

Only the files this feature uses.

```
backend/app/
├── api/routes/settings.py     preferences, consents (+ history), about, export, password,
│                              deactivate, concerns, health-centre
└── services/consent.py        current_consents(), record_consents()
frontend/src/
├── pages/Settings.tsx         all tabs: AccountTab, PreferencesTab, PrivacyTab, SupportTab,
│                              CentreTab, WorkTab
└── services/account.ts        one function per settings endpoint, downloadExport()
```

How the pieces connect

1. Each tab loads its own data with `useResource` and saves with one call from `services/account.ts`.
2. Preferences are saved as a **partial** update: only the changed keys are sent and merged on the server.
3. Consent changes append rows; turning health tracking off also updates the user, which blocks new readings (guide 01).

Code walkthrough

Every excerpt below is copied straight from the file, with its real line numbers, when this guide was generated.

1. `backend/app/api/routes/settings.py` · _deep_merge

```
89  def _deep_merge(base: dict[str, Any], patch: dict[str, Any]) -> dict[str, Any]:
90      merged = copy.deepcopy(base)
91      for key, value in patch.items():
92          if isinstance(value, dict) and isinstance(merged.get(key), dict):
93              merged[key] = _deep_merge(merged[key], value)
94          else:
95              merged[key] = value
96      return merged
```

What the code means

- Merges a partial change into the stored preferences, including nested objects such as `working_hours`.
- `copy.deepcopy(base)` copies first, so the original is never changed by accident.
- It calls itself for nested dictionaries: a **recursive** function.

2. `backend/app/api/routes/settings.py` · update_preferences

```
113 @router.patch("/preferences", response_model=Preferences)
114 async def update_preferences(
115     patch: dict[str, Any], user: User = Depends(require_any_verified), db: AsyncSession = Depends(get_sessio
        n)
116 ) -> Preferences:
117     """Any subset, nested keys included. The merged result is validated as a whole."""
118     merged = _deep_merge(_current_preferences(user).model_dump(), patch)
119     try:
120         prefs = Preferences.model_validate(merged)
121     except ValidationError as exc:
122         raise RequestValidationError(exc.errors(include_context=False, include_input=False)) from None
123     user.preferences = prefs.model_dump()  # a new object, so the JSON column is marked dirty
124     await db.commit()
125     return prefs
```

What the code means

- `patch: dict[str, Any]` accepts any JSON object; the merged result is then validated as a whole by `Preferences.model_validate`, so a patch cannot leave invalid settings.
- `RequestValidationError(...)` returns the same 422 format as any other bad request.
- `user.preferences = prefs.model_dump()` assigns a **new** object, so SQLAlchemy notices the JSON column changed.

3. `backend/app/services/consent.py` · record_consents

```
22 async def record_consents(
23     session: AsyncSession,
24     user_id: uuid.UUID,
25     choices: Mapping[ConsentType, bool],
26     *,
27     source: str,
28     ip_address: str | None,
29 ) -> list[ConsentRecord]:
30     """Append a row for each choice that differs from the current state.
31 
32     A first answer is always recorded, including a refusal: "asked and declined"
33     is history worth keeping. Does not commit; the caller owns the transaction.
34     """
35     current = await current_consents(session, user_id)
36     added = [
37         ConsentRecord(user_id=user_id, consent_type=consent_type, granted=granted, source=source, ip_address
            =ip_address)
38         for consent_type, granted in choices.items()
39         if current.get(consent_type) != granted
40     ]
41     session.add_all(added)
42     return added
```

What the code means

- Only choices that differ from the current state are added, so saving the same answer twice adds nothing.
- A first answer is always recorded, even "no": "asked and declined" is history worth keeping.
- It does not commit; the route that calls it decides when to save.

4. `backend/app/api/routes/settings.py` · update_consents

```python
@router.patch("/consents", response_model=ConsentsOut)
async def update_consents(
    data: ConsentUpdate,
    request: Request,
    user: User = Depends(require_any_verified),
    db: AsyncSession = Depends(get_session),
) -> ConsentsOut:
    choices = {c.consent_type: c.granted for c in data.consents}
    await record_consents(db, user.id, choices, source="settings", ip_address=sessions.client_ip(request))
    if ConsentType.health_tracking in choices and user.role == UserRole.user:
        granted = choices[ConsentType.health_tracking]
        user.consent = granted
        profile = await db.scalar(select(UserProfile).where(UserProfile.user_id == user.id))
        if profile is not None:
            profile.permission_health_tracking = granted
    await db.commit()
    return await _consents_out(db, user)
```

What the code means

- `{c.consent_type: c.granted for c in data.consents}` turns the list into a dictionary.
- Health tracking is also stored on the user (`user.consent`) and the profile, because logging vitals checks it there.

5. `backend/app/api/routes/settings.py` · change_health_centre

```python
@router.patch("/health-centre", response_model=HealthCentreOut)
async def change_health_centre(
    data: HealthCentreChange,
    request: Request,
    user: User = Depends(require_onboarding_user),
    db: AsyncSession = Depends(get_session),
) -> HealthCentreOut:
    if not user.district or not await facility_serves_district(db, data.phc_id, user.district, user.state):
        raise HTTPException(
            status.HTTP_422_UNPROCESSABLE_ENTITY, "Choose an active health centre in your own district."
        )
    if data.phc_id == user.phc_id:
        return await _health_centre(db, user)
    # Visits booked by the old centre's workers can no longer happen: they lose sight of this person.
    open_visits = await db.execute(
        select(ScheduledVisit, User)
        .join(User, User.id == ScheduledVisit.worker_id)
        .where(
            ScheduledVisit.citizen_id == user.id,
            ScheduledVisit.status == VisitStatus.scheduled,
            User.phc_id != data.phc_id,
        )
    )
    for visit, worker in open_visits.all():
        visit.status = VisitStatus.cancelled
        notify_changed(db, visit, user, worker, "cancelled")
    user.phc_id = data.phc_id
    db.add(AuditLog(user_id=user.id, action="change_health_centre", entity="valid_phc_facilities",
                    entity_id=data.phc_id, ip_address=sessions.client_ip(request) or "unknown"))
    await db.commit()
    return await _health_centre(db, user)
```

What the code means

- A citizen may only choose an active centre in their own district (`facility_serves_district`).
- Visits booked by the old centre's workers are cancelled and those workers are told, because after the switch they can no longer see this person.

6. `frontend/src/pages/Settings.tsx` · Toggle

```
22  function Toggle({ checked, onChange, label, hint, disabled }: { checked: boolean; onChange: (v: boolean) =>
        void; label: ReactNode; hint?: ReactNode; disabled?: boolean }) {
23    return (
24      <label className="flex items-start gap-3 py-2">
25        <input type="checkbox" className="mt-1" checked={checked} disabled={disabled} onChange={(e) => onChang
              e(e.target.checked)} />
26        <span className="text-sm">
27          <span className="font-medium text-ink">{label}</span>
28          {hint && <span className="block text-ink-soft">{hint}</span>}
29        </span>
30      </label>
31    );
32  }
```

What the code means

- A reusable checkbox with a label and a hint. `onChange(e.target.checked)` passes true/false up to the parent.
- `hint?: ReactNode` — the `?` makes the prop optional; `ReactNode` means "anything React can draw".

7. `frontend/src/pages/Settings.tsx` · PrivacyTab (loading and saving)

```
178  function PrivacyTab() {
179    const { user, refreshUser } = useAuth();
180    const consents = useResource(() => Promise.all([account.getConsents(), account.getConsentHistory(), accoun
           t.getAbout()]), "consents");
181    const [err, setErr] = useState<string | null>(null);
182    const [downloading, setDownloading] = useState(false);
183
184    if (consents.error) return <ErrorNote message={consents.error} onRetry={consents.reload} />;
185    if (!consents.data) return <Loading />;
186    const [current, history, about] = consents.data;
187    const types = (Object.keys(CONSENT_TEXT) as ConsentType[]).filter((t) => user?.role === "user" || !["healt
           h_tracking", "phc_worker_access", "community_surveillance"].includes(t));
188
189    const change = async (type: ConsentType, granted: boolean) => {
190      setErr(null);
191      try {
192        await account.updateConsent(type, granted);
193        if (type === "health_tracking") await refreshUser();
194        consents.reload();
195      } catch (e) {
196        setErr(errorMessage(e));
197      }
198    };
```

What the code means

- Loads current consents, history and the "about" texts together.
- `types` hides citizen-only consents from workers.
- `change` saves one consent; after a health-tracking change it calls `refreshUser()` so the rest of the app sees it, then reloads the tab.

8. `frontend/src/services/account.ts` · downloadExport

```
 97  export async function downloadExport(): Promise<void> {
 98    const response = await api.get<Blob>("/settings/export", { responseType: "blob" });
 99    const url = URL.createObjectURL(response.data);
100    const link = document.createElement("a");
101    link.href = url;
102    link.download = `healthnexus-export-${new Date().toISOString().slice(0, 10)}.json`;
103    link.click();
104    URL.revokeObjectURL(url);
105  }
```

What the code means

- `responseType: "blob"` asks axios for the raw file.
- `URL.createObjectURL` gives it a temporary link; clicking a hidden `<a download>` saves it as `healthnexus-export-<date>.json`; `revokeObjectURL` frees the memory.

Syntax used in this feature

Code	What it means
`copy.deepcopy(x)`	A full independent copy, nested parts included.
`Model.model_validate(d)`	Check a dictionary against a Pydantic model.
`{k: v for c in xs}`	Dictionary comprehension.
`prop?: Type`	An optional prop.
`URL.createObjectURL(blob)`	A temporary link to data in the browser.